

```

// SpectrumWidget.cpp — constructor
SpectrumWidget::SpectrumWidget(QWidget *parent) : QWidget(parent)
{
    // ... existing setup (customPlot, graph(0), axis labels, etc.)

    plotTimer = new QTimer(this);
    plotTimer->setInterval(33); // ~30 fps, plenty for visual smoothness
    connect(plotTimer, &QTimer::timeout, this, &SpectrumWidget::onPlotTimerTick);
    plotTimer->start();
}

void SpectrumWidget::setStartStop(double startFreq, double stopFreq)
{
    if (startFreq == m_startFreq && stopFreq == m_stopFreq)
        return;

    m_startFreq = startFreq;
    m_stopFreq = stopFreq;

    freq.clear();
    power.clear();
    customPlot->graph(0)->data()->clear();

    customPlot->xAxis->setRange(m_startFreq, m_stopFreq);
    customPlot->replot();
}

// Called extremely frequently — keep this as cheap as possible, NO replot here
void SpectrumWidget::updateSpectrumData(const QVector<double> &freqMHz, const
QVector<double> &powerdBm)
{
    if (freqMHz.isEmpty())
        return;

    freq += freqMHz;
    power += powerdBm;

    // Detect end of sweep: wrap buffer once stop freq is reached
    if (freq.last() >= m_stopFreq - 1.0)
    {
        // Keep this completed sweep as-is; next call starts a fresh buffer
        // (timer will still pick up and draw what's here before it's cleared)
        dataChanged = true;
        onPlotTimerTick(); // force one immediate draw of the completed sweep
        freq.clear();
        power.clear();
        return;
    }

    dataChanged = true; // just mark dirty, timer will pick it up
}

// Runs on a fixed schedule, independent of data arrival rate
void SpectrumWidget::onPlotTimerTick()
{
    if (!dataChanged || freq.isEmpty())
        return;

```

```

customPlot->graph(0)->setData(freq, power, true);
customPlot->replot(QCustomPlot::rpQueuedReplot);
dataChanged = false;
}

```

```

// SpectrumWidget.h
private:
    QVector<double> freq, power;    // currently-filling sweep buffer
    double m_startFreq = -1;
    double m_stopFreq = -1;
    QTimer *plotTimer;
    bool dataChanged = false;      // dirty flag, avoid replotting when nothing new arrived

public:
    void setStartStop(double startFreq, double stopFreq);

private slots:
    void onPlotTimerTick();

```

```

// SpectrumWidget.h

```

```

class SpectrumWidget : public QWidget
{
    Q_OBJECT

public:
    explicit SpectrumWidget(QWidget *parent = nullptr);

    void setStartStop(double startFreq,
                      double stopFreq);

    void updateSpectrumData(const QVector<double> &freqMHz,
                           const QVector<double> &powerdBm);

private slots:
    void onPlotTimer();

private:
    QCustomPlot *customPlot;

    QVector<double> freq;
    QVector<double> power;

    double m_startFreq = 20;
    double m_stopFreq = 6000;

    int expectedBins = 0;
    int receivedBins = 0;

    bool dataChanged = false;
    bool sweepComplete = false;

```

```

    QTimer *plotTimer;
};

// SpectrumWidget.cpp

SpectrumWidget::SpectrumWidget(QWidget *parent)
    : QWidget(parent)
{
    customPlot = new QCustomPlot(this);

    customPlot->addGraph();

    plotTimer = new QTimer(this);

    plotTimer->setInterval(33);    // 30 FPS

    connect(plotTimer,
            &QTimer::timeout,
            this,
            &SpectrumWidget::onPlotTimer);

    plotTimer->start();
}

void SpectrumWidget::setStartStop(double startFreq,
                                   double stopFreq)
{
    m_startFreq = startFreq;
    m_stopFreq = stopFreq;

    freq.clear();
    power.clear();

    receivedBins = 0;
    sweepComplete = false;

    // Number of 20 MHz bins

    expectedBins =
        ((m_stopFreq - m_startFreq) / 20) + 1;

    // Reserve memory only once

    int totalPoints = expectedBins * 1280;

    freq.reserve(totalPoints);
    power.reserve(totalPoints);

    customPlot->graph(0)->data()->clear();

    customPlot->xAxis->setRange(m_startFreq,
                               m_stopFreq);

    customPlot->replot();
}

```

```

/*=====
=====
SpectrumWidget::SpectrumWidget(QWidget *parent)
: QWidget(parent)
{
    customPlot = new QCustomPlot(this);

    QVBoxLayout *layout = new QVBoxLayout(this);
    layout->addWidget(customPlot);

    customPlot->addGraph();

    //-----
    // Enable Mouse Interaction
    //-----

    customPlot->setInteractions(
        QCP::iRangeDrag |
        QCP::iRangeZoom |
        QCP::iSelectPlottables);

    customPlot->axisRect()->setRangeDrag(Qt::Horizontal);
    customPlot->axisRect()->setRangeZoom(Qt::Horizontal);

    //-----
    // Vertical Marker
    //-----

    verticalLine = new QCPLItemStraightLine(customPlot);

    verticalLine->setPen(QPen(Qt::yellow,2));

    verticalLine->point1->setCoords(100,-200);
    verticalLine->point2->setCoords(100,100);

    //-----
    // Horizontal Marker
    //-----

    horizontalLine = new QCPLItemStraightLine(customPlot);

    horizontalLine->setPen(QPen(Qt::green,2));

    horizontalLine->point1->setCoords(0,-50);
    horizontalLine->point2->setCoords(6000,-50);

    //-----
    // Crosshair
    //-----

```

```

tracer = new QCItemTracer(customPlot);

tracer->setGraph(customPlot->graph(0));

tracer->setStyle(QCItemTracer::tsCrosshair);

tracer->setPen(QPen(Qt::red));

tracer->setBrush(Qt::red);

tracer->setSize(10);

//-----

connect(customPlot,
        &QCustomPlot::mouseMove,
        this,
        &SpectrumWidget::onMouseMove);

connect(customPlot,
        &QCustomPlot::mousePress,
        this,
        &SpectrumWidget::onMousePress);
}

void SpectrumWidget::updateSpectrumData(const QVector<double> &freqMHz,
                                         const QVector<double> &powerdBm)
{
    if(freqMHz.isEmpty())
        return;

    // Store incoming points

    freq += freqMHz;
    power += powerdBm;

    receivedBins++;

    dataChanged = true;

    // Last 20 MHz block received

    if(receivedBins >= expectedBins)
    {
        sweepComplete = true;
        receivedBins = 0;
    }
}

void SpectrumWidget::onPlotTimer()
{
    if(!dataChanged)
        return;

    customPlot->graph(0)->setData(freq,
                                   power,

```

```

        true);

    customPlot->replot(QCustomPlot::rpQueuedReplot);

    dataChanged = false;

    // One complete sweep finished

    if(sweepComplete)
    {
        freq.clear();
        power.clear();

        customPlot->graph(0)->data()->clear();

        sweepComplete = false;
    }
}
}

```

```

#include "qcustomplot.h"

class SpectrumWidget : public QWidget
{
    Q_OBJECT

public:
    explicit SpectrumWidget(QWidget *parent = nullptr);

private slots:
    void onMouseMove(QMouseEvent *event);
    void onMousePress(QMouseEvent *event);

private:
    QCustomPlot *customPlot;

    QCPLItemStraightLine *verticalLine;
    QCPLItemStraightLine *horizontalLine;

    QCPLItemTracer *tracer;

    QVector<double> freq;
    QVector<double> power;
};

```

```

#include "waterfalldata.h"
#include <QPainter>
#include <QDebug>
#include <QMouseEvent>
#include <QDateTime>
#include <cstring> // for memcpy

WaterfallData::WaterfallData(QWidget *parent)
    : QWidget(parent),
      m_image(512, 400, QImage::Format_RGB32) // initial placeholder image, resized later to match
      real bin count

```

```

{
    m_image.fill(Qt::black);          // start with a blank black waterfall
    setMouseTracking(true);           // IMPORTANT: get mouseMoveEvent without a button held
down
}

    m_updateTimer = new QTimer(this); // timer drives row-by-row rendering from the queue
    connect(m_updateTimer, &QTimer::timeout, this, &WaterfallData::processQueue); // pull one
spectrum per tick
    m_updateTimer->start(30);          // default 30ms refresh interval
}

void WaterfallData::setUpdateInterval(int ms)
{
    m_updateTimer->setInterval(qMax(1, ms)); // change refresh rate, minimum 1ms guard
}

// Connect your source's signal (e.g. signalStartStop(double,double)) to this slot.
void WaterfallData::setStartStopFreq(double startFreq, double stopFreq)
{
    bool rangeChanged = (startFreq != m_startFreq) || (stopFreq != m_stopFreq); // detect if range
actually changed

    m_startFreq = startFreq; // store new left edge of X axis
    m_stopFreq = stopFreq;   // store new right edge of X axis

    if (rangeChanged)
        clearWaterfall(); // wipe old image + queue since old pixels no longer match new axis

    update(); // repaint immediately so axis labels reflect new range right away
}

void WaterfallData::clearWaterfall()
{
    m_image.fill(Qt::black); // erase all previously drawn waterfall rows
    m_spectrumQueue.clear(); // drop any pending spectra still queued from the old range
    update(); // trigger repaint with blank waterfall
}

void WaterfallData::addSpectrum(const QVector<double> &power)
{
    if (power.isEmpty())
        return; // ignore empty spectra, nothing to draw

    if (power.size() != m_spectrumSize)
    {
        m_spectrumQueue.clear(); // old queued rows have a different bin count, can't mix
them
        m_spectrumSize = power.size(); // remember new bin count as the current baseline
    }

    m_spectrumQueue.enqueue(power); // add this spectrum to the history queue

    while (m_spectrumQueue.size() > m_maxQueueSize)
        m_spectrumQueue.dequeue(); // trim oldest entries beyond configured history
depth
}

```

```

void WaterfallData::processQueue()
{
    if (m_spectrumQueue.isEmpty())
        return; // nothing new to render this tick

    QVector<double> power = m_spectrumQueue.dequeue(); // take oldest queued spectrum first
(FIFO)
    renderSpectrum(power); // draw it into the waterfall image
}

void WaterfallData::setHistoryDepth(int rows)
{
    m_maxQueueSize = qMax(1, rows); // update how many rows of history to retain,
minimum 1
    while (m_spectrumQueue.size() > m_maxQueueSize)
        m_spectrumQueue.dequeue(); // immediately trim if new depth is smaller than
current queue
}

// renderSpectrum no longer hardcodes start/stop freq - it's set externally via setStartStopFreq().
void WaterfallData::renderSpectrum(const QVector<double> &power)
{
    // Width is resized to match however many frequency bins the incoming spectrum has;
    // height stays fixed at 500px regardless of widget size.
    if (m_image.width() != power.size())
    {
        m_image = QImage(power.size(), 500, QImage::Format_RGB32); // recreate image at new bin-
count width, fixed 500px height
        m_image.fill(Qt::black); // clear new image so no garbage pixels show
    }

    const int rowHeight = m_frameHeight; // pixel height of one new spectrum row

    for (int y = m_image.height() - 1; y >= rowHeight; --y)
        memcpy(m_image.scanLine(y),
            m_image.scanLine(y - rowHeight),
            m_image.bytesPerLine()); // shift all existing rows down by rowHeight (scroll effect)

    QVector<QRgb> colors(power.size()); // buffer to hold this row's pixel colors
    for (int x = 0; x < power.size(); ++x)
        colors[x] = powerToColor(power[x]).rgb(); // convert each dBm value into its display color

    for (int y = 0; y < rowHeight; ++y)
    {
        QRgb *line = reinterpret_cast<QRgb *>(m_image.scanLine(y)); // pointer to the new top row(s)
        memcpy(line, colors.constData(), colors.size() * sizeof(QRgb)); // write new spectrum into the
top row
    }

    update(); // repaint widget with the updated image
}

QColor WaterfallData::powerToColor(double power)
{
    // Match the same dynamic range used elsewhere (SpectrumWidget: -150..0 dBm).
    power = qBound(kMinPowerDbm, power, kMaxPowerDbm); // clamp input so out-of-range
values don't break the mapping

```



```

    double t = (power - kMinPowerDbm) / (kMaxPowerDbm - kMinPowerDbm); // normalize power
    to 0.0-1.0

    int r = int(255 * t);                // red rises with power
    int g = int(255 * (1.0 - std::abs(t - 0.5) * 2.0)); // green peaks at mid-range power
    int b = int(255 * (1.0 - t));        // blue falls as power rises

    return QColor(r, g, b);              // final RGB color for this power value
}

// Vertical color-scale legend (like a standalone dBm colorbar), drawn on the
// left edge of the widget, aligned top-to-bottom with the plot area.
void WaterfallData::drawColorScale(QPainter &painter, const QRect &plotRect)
{
    const int barX = 5;                  // fixed left offset for the color bar
    const int colorBarWidth = 15;        // fixed width of the color bar

    QRect colorRect(barX, plotRect.top(), colorBarWidth, plotRect.height()); // bar spans full plot
    height

    QLinearGradient gradient(colorRect.topLeft(), colorRect.bottomLeft()); // gradient runs top (max)
    to bottom (min)
    const int stops = 8;                  // number of gradient sample points
    for (int i = 0; i <= stops; ++i)
    {
        double t = double(i) / stops;    // 0 = top, 1 = bottom
        double power = kMaxPowerDbm - t * (kMaxPowerDbm - kMinPowerDbm); // corresponding
        dBm value at this stop
        gradient.setColorAt(t, powerToColor(power)); // sample the real color mapping for accuracy
    }

    painter.setBrush(gradient);            // fill bar with the gradient
    painter.setPen(Qt::NoPen);              // no border for the fill pass
    painter.drawRect(colorRect);            // draw the filled color bar

    painter.setPen(Qt::gray);              // outline color for the bar border
    painter.setBrush(Qt::NoBrush);         // no fill for the outline pass
    painter.drawRect(colorRect);            // draw border around the bar

    painter.setPen(Qt::white);              // tick label color
    painter.setFont(QFont("Arial", 8, QFont::Bold)); // tick label font

    const int stepDb = 10;                  // label every 10 dB
    for (int value = int(kMaxPowerDbm); value >= int(kMinPowerDbm); value -= stepDb)
    {
        double t = (kMaxPowerDbm - value) / (kMaxPowerDbm - kMinPowerDbm); // position ratio for
        this dB value
        int y = colorRect.top() + int(t * colorRect.height()); // pixel Y for this tick
        painter.drawText(barX + colorBarWidth + 4, y + 4, QString::number(value)); // draw the dB
        number beside the bar
    }
}

// Shared so mouseMoveEvent and paintEvent always agree on where the plot lives.
QRect WaterfallData::plotRectGeometry() const
{
    const int leftMargin = 60;              // space reserved for color scale + Y labels

```

```

const int topMargin = 20;           // top padding
const int bottomMargin = 30;        // space reserved for X axis labels
return QRect(leftMargin,
              topMargin,
              width() - leftMargin - 10, // plot width shrinks with right margin of 10px
              height() - topMargin - bottomMargin); // plot height fills remaining widget space
}

void WaterfallData::mouseMoveEvent(QMouseEvent *event)
{
    QRect plotRect = plotRectGeometry(); // get current plot area
    m_mousePos = event->pos();           // store latest cursor position
    m_mouseInPlot = plotRect.contains(m_mousePos); // check if cursor is inside the plot area
    update();                           // repaint so the readout/crosshair follows the cursor
    QWidget::mouseMoveEvent(event);     // pass event up the chain
}

void WaterfallData::leaveEvent(QEvent *event)
{
    m_mouseInPlot = false; // hide readout once cursor leaves the widget
    update();              // repaint to remove the readout/crosshair
    QWidget::leaveEvent(event); // pass event up the chain
}

void WaterfallData::paintEvent(QPaintEvent *)
{
    QPainter painter(this);

    painter.fillRect(rect(), Qt::black); // fill whole widget background black first

    QRect plotRect = plotRectGeometry(); // compute plot area (depends on widget size only)

    painter.drawImage(plotRect, m_image); // draw the waterfall image scaled into the plot
    rectangle

    drawColorScale(painter, plotRect); // draw the dBm color-scale legend on the left

    painter.setPen(Qt::white); // set pen for axis lines/labels

    painter.drawLine(plotRect.bottomLeft(), plotRect.bottomRight()); // X axis baseline (position
    fixed to plotRect, not frequency)

    painter.drawLine(plotRect.topLeft(), plotRect.bottomLeft()); // Y axis baseline

    painter.drawText(plotRect.left() - 10,
                     plotRect.bottom() + 20,
                     QString::number(m_startFreq, 'f', 3)); // start frequency label, live from
    m_startFreq

    painter.drawText(plotRect.right() - 40,
                     plotRect.bottom() + 20,
                     QString::number(m_stopFreq, 'f', 3)); // stop frequency label, live from
    m_stopFreq

    double center = (m_startFreq + m_stopFreq) / 2.0; // center frequency recomputed
    every paint

```

```

painter.drawText(plotRect.center().x() - 20,
    plotRect.bottom() + 20,
    QString::number(center, 'f', 3));    // center frequency label

const int rowHeight = qMax(1, m_frameHeight);    // pixel height of one waterfall row
const int intervalMs = qMax(1, m_updateTimer->interval()); // time between rows, from the
render timer
const QDateTime now = QDateTime::currentDateTime(); // reference "now" for row 0
(topmost/newest)

painter.setFont(QFont("Arial", 7));    // small font for row timestamps

int rowsVisible = plotRect.height() / rowHeight;    // how many timestamp rows fit on screen
for (int i = 0; i < rowsVisible; ++i)
{
    int y = plotRect.top() + i * rowHeight;    // vertical pixel position of this row
    if (y > plotRect.bottom())
        break;    // stop once past the bottom of the plot

    QDateTime rowTime = now.addMSecs(-qint64(i) * intervalMs); // time this row represents
    QString label = rowTime.toString("hh:mm:ss.zzz");    // formatted timestamp text

    QRect rowLabelRect(plotRect.left() + 2, y, 90, rowHeight); // background box position for this
row
    painter.fillRect(rowLabelRect, QColor(0, 0, 0, 90));    // translucent background for
readability

    painter.setPen(Qt::white);    // white text for timestamp
    painter.drawText(rowLabelRect, Qt::AlignVCenter | Qt::AlignLeft, label); // draw the timestamp
}

if (m_mouseInPlot)
{
    double xRatio = double(m_mousePos.x() - plotRect.left()) / plotRect.width(); // cursor X as ratio
across plot
    double freq = m_startFreq + xRatio * (m_stopFreq - m_startFreq);    // map ratio to
current frequency range

    double yRatio = double(m_mousePos.y() - plotRect.top()) / plotRect.height(); // cursor Y as
ratio down plot
    int rowIndex = int(yRatio * m_image.height());    // map ratio to image row
index
    rowIndex = qBound(0, rowIndex, m_image.height() - 1);    // clamp to valid row
range

    QString label = QString("x:%1 y:%2 Freq:%3")
        .arg(m_mousePos.x())
        .arg(m_mousePos.y())
        .arg(freq, 0, 'f', 3);    // readout text with live frequency

    QFontMetrics fm(painter.font());    // metrics for sizing the readout box
    int textWidth = fm.horizontalAdvance(label) + 10;    // box width based on text + padding
    int textHeight = fm.height() + 6;    // box height based on text + padding

    int boxX = qMin(m_mousePos.x() + 12, width() - textWidth - 2); // keep box inside widget
horizontally
    int boxY = qMax(m_mousePos.y() - textHeight - 4, 0);    // keep box inside widget vertically

```

```

        QRect labelRect(boxX, boxY, textWidth, textHeight);        // final readout box rectangle

        painter.fillRect(labelRect, QColor(0, 0, 0, 180));    // semi-transparent background for readout
        painter.setPen(Qt::yellow);        // yellow text for visibility
        painter.drawText(labelRect, Qt::AlignCenter, label);    // draw the readout text

        painter.setPen(QPen(Qt::gray, 1, Qt::DashLine));    // dashed gray pen for crosshair
        painter.drawLine(plotRect.left(), m_mousePos.y(), plotRect.right(), m_mousePos.y()); //
horizontal crosshair line
        painter.drawLine(m_mousePos.x(), plotRect.top(), m_mousePos.x(), plotRect.bottom()); //
vertical crosshair line
    }
}

```

*****=====

```
#pragma once
```

```

#include <QWidget>
#include <QImage>
#include <QVector>
#include <QQueue>
#include <QTimer>
#include <QMouseEvent>
#include <QDateTime>
class WaterfallData : public QWidget
{
    Q_OBJECT
public:
    explicit WaterfallData(QWidget *parent = nullptr);
    void addSpectrum(const QVector<double> &power);
    void setFrameHeight(int px);
    void setUpdateInterval(int ms);

```

```

static constexpr double kMinPowerDbm = -150.0;
static constexpr double kMaxPowerDbm = 0.0;
public slots:
    void setStartStopFreq(double startFreq, double stopFreq);
        // QColor powerToColor(double power);
    void setHistoryDepth(int row);
protected:
    void paintEvent(QPaintEvent *) override;
    void mouseMoveEvent(QMouseEvent *event) override;
    void leaveEvent(QEvent *event) override;

private slots:
    void processQueue();
    void clearWaterfall();

    void drawColorScale(QPainter &painter, const QRect &plotRect);
    // QStringList m_timeHistory;
    // void drawColorScale(QPainter &painter, const QRect &plotRect);
private:
    QColor powerToColor(double power);
    void renderSpectrum(const QVector<double> &power);
    QRect plotRectGeometry() const;

    QImage m_image;
    double m_startFreq = 20.0, m_stopFreq = 6000.0;
    int m_frameHeight = 40;
    QQueue<QVector<double>> m_spectrumQueue;
    QTimer *m_updateTimer = nullptr;
    int m_spectrumSize = 0; // number of bins in the most recent spectrum (1..850)
    int m_maxQueueSize = 200; // fixed history depth (number of waterfall rows to retain)
    bool m_mouseInPlot = false;
    QPoint m_mousePos;
};

```